Project — до 14 июня 2026 г. 23:59 MSK

Итоговый проект: Letter Classifier (EMNIST Letters)

Цель

В этом задании вы разработаете систему распознавания рукописных букв английского алфавита на основе датасета EMNIST Letters. Вы реализуете полный цикл создания модели машинного обучения — от загрузки данных до инференса на пользовательских примерах. В ходе выполнения проекта вы закрепите навыки работы с двумя основными фреймворками глубокого обучения — PyTorch и TensorFlow — и научитесь сравнивать их между собой.

1. Описание датасета

> **EMNIST Letters** — это расширенная версия MNIST, содержащая рукописные буквы английского алфавита.

Датасет включает 145 600 изображений размером 28 × 28 пикселей в градациях серого. Из них 124 800 изображений предназначены для обучения и 20 800 — для тестирования. Каждое изображение относится к одному из 26 классов, соответствующих буквам от A до Z. В исходных данных метки представлены числами от 1 до 26, где 1 соответствует букве A, 2 — B и так далее до 26 — Z.

Датасет доступен для загрузки двумя способами:

- PyTorch: используется встроенная функция torchvision.datasets.EMNIST с параметром split='letters'.
- TensorFlow: используется библиотека tensorflow_datasets с командой tfds.load('emnist/letters').

Оба способа являются официальными и автоматически скачивают данные при первом запуске.

2. Общие требования к выполнению

Формат сдачи

Проект должен быть сдан в виде одного Jupyter Notebook с именем «Фамилия_Имя_LetterClassifier.ipynb». Ноутбук должен содержать весь код и все выводы: графики, таблицы, текстовые ячейки с анализом. Код должен быть полностью воспроизводимым — при последовательном запуске всех ячеек должно получаться то же самое решение.

Требования к коду

Код должен быть читаемым и структурированным. Используйте осмысленные имена переменных, добавляйте комментарии к сложным участкам. Разбивайте длинные ячейки на логические блоки. Для воспроизводимости результатов зафиксируйте random seed в обоих фреймворках.

Требования к выводам и анализу

Каждый этап должен сопровождаться текстовыми ячейками с анализом полученных результатов. Все графики должны иметь заголовки, подписи осей и легенду там, где это необходимо. Выводы должны быть аргументированными и должны опираться на конкретные численные результаты.

Установка необходимых библиотек

Перед началом работы установите все необходимые библиотеки. Выполните следующую команду в терминале или в первой ячейке ноутбука:

```
! pip install torch torchvision tensorflow matplotlib
numpy scikit-learn pillow
```

3. Инструкции

Загрузка и подготовка данных

1. Загрузка датасета в PyTorch — через torchvision.

В PyTorch датасет EMNIST Letters загружается с помощью модуля `torchvision.datasets`. При загрузке необходимо указать параметр split='letters', чтобы получить именно буквенный датасет, а не цифры или другие варианты EMNIST.

Параметр download=True обеспечивает автоматическую загрузку данных при первом запуске. Для предобработки используется transforms.Compose, который последовательно применяет преобразования: сначала ToTensor() преобразует PIL-изображение в тензор и нормализует пиксели к диапазону [0, 1], затем Lambda изменяет форму тензора с (1, 28, 28) на (784) — это необходимо для подачи в полносвязную сеть.

```
# импорт необходимых библиотек для PyTorch
import torch
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split


# определяем преобразования для изображений
transform = transforms.Compose([
    transforms.ToTensor(),                          # PIL
→ тензор, значения [0, 1]
    transforms.Lambda(lambda x: x.view(-1))         #
(1,28,28) → (784)
])


# загрузка обучающей выборки
full_train = datasets.EMNIST(
    root='./data',          # папка для сохранения данных
    split='letters',        # буквенный датасет (не
digits или balanced)
    train=True,             # обучающая выборка
    download=True,          # скачать, если нет локально
    transform=transform     # применяем преобразования
)


# загрузка тестовой выборки
test_dataset = datasets.EMNIST(
    root='./data',
    split='letters',
    train=False,            # тестовая выборка
    download=True,
    transform=transform
)


print(f"Размер полной обучающей выборки:
{len(full_train)}")
print(f"Размер оригинальной тестовой выборки:
{len(test_dataset)}")
```

2. Загрузка датасета в TensorFlow — через конвертацию из PyTorch.

Для загрузки данных в TensorFlow используйте данные, полученные из PyTorch. Это гарантирует, что вы работаете с правильным датасетом EMNIST Letters (26 классов, 124800/20800 изображений). Преобразуйте тензоры PyTorch в NumPy-массивы, выполните нормализацию и создайте tf.data.Dataset.

```
import tensorflow as tf
import numpy as np


# получение данных из PyTorch
X_train = full_train.data.numpy().reshape(-1,
784).astype('float32') / 255.0
y_train = full_train.targets.numpy().astype('int32') - 1


X_test = test_dataset.data.numpy().reshape(-1,
784).astype('float32') / 255.0
y_test = test_dataset.targets.numpy().astype('int32') - 1


print(f"X_train: {X_train.shape}, y_train:
{y_train.shape}")
print(f"X_test: {X_test.shape}, y_test: {y_test.shape}")


# создание tf.data.Dataset
BATCH_SIZE = 64
train_dataset =
tf.data.Dataset.from_tensor_slices((X_train, y_train))
train_dataset =
train_dataset.shuffle(1000).batch(BATCH_SIZE).prefetch(tf.
data.AUTOTUNE)


test_dataset = tf.data.Dataset.from_tensor_slices((X_test,
y_test))
test_dataset = test_dataset.batch(BATCH_SIZE)
```

Исследовательский анализ данных

После загрузки данных выполните их визуальное и количественное исследование.

Выведите общую информацию:

- Количество изображений в обучающей выборке.
- Количество изображений в тестовой выборке.
- Размер одного изображения в пикселях.
- Диапазон значений пикселей (минимальное и максимальное значение).

Постройте гистограмму распределения классов:

- Для обучающей выборки постройте гистограмму, показывающую, сколько примеров каждого класса (буквы A–Z) содержится в данных.
- Сделайте вывод: сбалансирован ли датасет? Есть ли буквы, которых значительно больше или меньше?

Визуализируйте примеры изображений:

- Для каждой буквы алфавита от A до Z выведите по пять случайных примеров.
- Изображения должны быть подписаны соответствующей буквой.
- Используйте Matplotlib для отображения с цветовой картой 'gray'.

Проверьте корректность предобработки:

- Убедитесь, что значения пикселей находятся в диапазоне [0, 1].
- Выведите минимальное и максимальное значения пикселя для одного из изображений.

Разбиение на train и validation

Из обучающей выборки выделите валидационную подвыборку для контроля качества в процессе обучения. Тестовая выборка остаётся нетронутой и будет использована только для финальной оценки модели.

PyTorch

Используйте torch.utils.data.random_split для разделения train_dataset_full на обучающую и валидационную выборки в соотношении 90 и 10%. Зафиксируйте random seed для воспроизводимости.

TensorFlow

Разделите NumPy-массивы в том же соотношении 90 и 10%.

Сравнение пайплайнов

Напишите текстовую ячейку, в которой сравните подходы к загрузке и подготовке данных в PyTorch и TensorFlow. Опишите, какой подход показался вам более интуитивным, какие сложности возникли и что можно было бы улучшить.

Baseline MLP-модели

Архитектура модели

Реализуйте многослойный перцептрон с одинаковой архитектурой для обоих фреймворков:

Слой	Тип	Вход	Выход	Активация
1	Полносвязный	784	512	ReLU
2	Полносвязный	512	256	ReLU
3	Полносвязный	256	128	ReLU
4	Полносвязный	128	26	Softmax

PyTorch — реализация и обучение

Создайте класс MLP, наследующий nn.Module. В конструкторе определите четыре полносвязных слоя с помощью nn.Linear. В методе forward реализуйте последовательное применение слоёв с активацией ReLU после первых трёх слоёв. Для обучения используйте функцию потерь nn.CrossEntropyLoss() и оптимизатор optim.Adam. Обучайте модель в течение десяти эпох с размером батча 64. После каждой эпохи вычисляйте accuracy на валидационной выборке. После завершения обучения вычислите accuracy на тестовой выборке и сохраните модель.

TensorFlow — реализация и обучение

Создайте модель с помощью tf.keras.Sequential. Добавьте входной слой с формой (784), три полносвязных слоя с 512, 256 и 128 нейронами и активацией ReLU и выходной слой с 26 нейронами и активацией Softmax. Скомпилируйте модель с оптимизатором Adam, функцией потерь sparse_categorical_crossentropy и метрикой accuracy. Обучите модель в течение десяти эпох с размером батча 64. После обучения вычислите accuracy на тестовой выборке и сохраните модель.

Сравнение результатов

Выполните сравнительный анализ моделей:

1. Постройте графики loss на обучении и validation accuracy для PyTorch и TensorFlow на одном рисунке. Графики должны быть подписаны и должны содержать легенду.
2. Сравните итоговую test accuracy, время обучения на одну эпоху и количество параметров в моделях.
3. Напишите вывод, в котором укажите, какой фреймворк дал лучшую accuracy, что было быстрее, какой фреймворк показался удобнее в реализации и почему результаты могли различаться.

Улучшение модели с помощью Dropout (необязательный пункт, но есть возможность получить бонусные баллы)

Добавление Dropout

Модифицируйте архитектуру моделей, добавив Dropout после каждого скрытого слоя с вероятностью отключения 0.3.

PyTorch: добавьте в класс MLP параметр dropout. В конструкторе создайте слой self.dropout = nn.Dropout(0.3), если dropout=True, иначе nn.Identity(). В методе forward применяйте dropout после каждого скрытого слоя. Создайте экземпляр модели с dropout=True и повторите обучение.

TensorFlow: добавьте слой tf.keras.layers.Dropout(0.3) после каждого полносвязного слоя. Создайте модель с dropout=True, скомпилируйте и обучите с теми же параметрами.

Сравнение с baseline

Для каждого фреймворка постройте графики training loss и validation accuracy для baseline- и dropout-моделей на одном рисунке. Сравните тестовую accuracy и разрыв между train- и val-метриками. Напишите вывод о том, помог ли Dropout справиться с переобучением.

Выбор лучшей модели

Для каждого фреймворка выберите модель (baseline или dropout) с наивысшей тестовой accuracy. Эти модели будут использоваться на этапе анализа ошибок.

Анализ ошибок

Confusion matrix

Для лучшей модели PyTorch и лучшей модели TensorFlow постройте confusion matrix. Получите предсказания на тестовой выборке. Постройте матрицу с помощью sklearn.metrics.confusion_matrix и визуализируйте её с подписанными осями и метками классов от A до Z. Добавьте цветовую шкалу. Проанализируйте, какие буквы распознаются лучше всего, а какие чаще всего путаются.

Топ-5 пар ошибок

На основе confusion matrix определите пять пар букв, которые модель путает чаще всего. Для каждой пары укажите истинную букву, предсказанную букву и количество ошибок.

Визуализация примеров ошибок

Найдите 10–15 примеров из тестовой выборки, где модель ошиблась. Для каждого примера выведите изображение буквы, подпишите истинную и предсказанную буквы, а также уверенность модели.

Сравнение ошибок двух фреймворков

Сравните confusion matrices PyTorch и TensorFlow. Есть ли различия в том, какие буквы путает каждая модель? Совпадают ли топ-5 пар ошибок? Проанализируйте причины ошибок. Напишите вывод — обобщите в нём основные паттерны ошибок и укажите, какой фреймворк допустил меньше ошибок.

Инференс и демо

Функция predict_letter

Напишите функцию predict_letter(image_path, model, framework), которая принимает путь к изображению, модель и указание фреймворка, а возвращает предсказанную букву и уверенность.

Внутри функции выполните следующие действия:

1. Загрузите изображение с помощью PIL.Image.open() и преобразуйте в grayscale.
2. Измените размер до 28 × 28 пикселей.
3. Преобразуйте в NumPy-массив типа float32.
4. Выполните инверсию цветов: если среднее значение пикселей больше 127 (светлый фон, тёмная буква), вычтите массив из 255.
5. Нормализуйте массив, разделив на 255.0.
6. Измените форму с (28, 28) на (1, 784).

Для предсказания:

- PyTorch: переведите модель в режим eval(), выполните forward pass, примените softmax, найдите класс с максимальной вероятностью, преобразуйте индекс в букву (0 → A).
- TensorFlow: выполните model.predict(), найдите класс с максимальной вероятностью, преобразуйте в букву.

Функция должна возвращать строку с заглавной буквой и значение уверенности от 0 до 1.

Тестирование на рукописных буквах

Создайте пять рукописных букв. Нарисуйте их в любом графическом редакторе или напишите на бумаге и сфотографируйте. Выберите буквы, которые могут быть сложными для распознавания, например O, Q, I, L, C, G. Сохраните изображения в формате PNG или JPG. Для каждого изображения вызовите функцию predict_letter с вашей лучшей моделью. Выведите результаты и визуализируйте изображения вместе с предсказаниями и уверенностью.

Отчёт о тестировании

Напишите текстовую ячейку с отчётом. Опишите, какие буквы вы нарисовали и каким был почерк. Проанализируйте результаты: какие буквы распознаны правильно, а какие нет, были ли неожиданные результаты. Объясните возможные причины ошибок. В завершение напишите общий вывод по проекту: что было самым сложным, какой фреймворк понравился больше и почему, что бы вы изменили, если бы делали проект снова.

4. Критерии оценки

Оценка за проект складывается из баллов за отдельные этапы. Всего за обязательную часть можно набрать **до 50 баллов.**

Каждый этап оценивается независимо: чем полнее и аккуратнее он выполнен, тем больше баллов вы получаете.

Вы можете получить **до 10 дополнительных баллов**, если улучшите модель с помощью Dropout. Важно:

- Это **необязательная часть.**
- Но она может помочь **компенсировать недочёты в других этапах.**

Основные критерии оценивания (максимум 50 баллов)

Этап 1. Data Pipeline (8 баллов)

Оба пайплайна реализованы корректно. EDA выполнен полностью (распределение классов, визуализация пяти примеров на букву). Разбиение train/val/test выполнено правильно для обоих фреймворков	8 баллов
Оба пайплайна работают, но есть мелкие недочёты. EDA выполнен частично	6 баллов
Корректно реализован только один пайплайн	4 балла
Данные не загружены, или пайплайны не работают	0 баллов

Этап 2: Baseline-модели (15 баллов)

Обе модели реализованы согласно архитектуре, обучены в течение 10 эпох. Проведено полное сравнение с графиками и выводами	15 баллов
Модели реализованы и обучены, но сравнение неполное	9 баллов
Модели реализованы, но не обучены или обучены с ошибками	5 баллов
Модели не реализованы	0 баллов

Dropout (дополнительное задание — 10 баллов)

Для обоих фреймворков добавлен Dropout, модели обучены. Проведено сравнение с baseline, выбрана лучшая модель	10 баллов
Dropout добавлен для обоих фреймворков, но сравнение неполное	7 баллов
Dropout добавлен только для одного фреймворка	5 баллов
Dropout не реализован	0 баллов

Этап 3. Анализ ошибок (12 баллов)

Confusion matrix построена для обоих фреймворков. Определены топ-5 пар ошибок. Визуализированы примеры ошибок. Проведён сравнительный анализ	12 баллов
Confusion matrix построена, но анализ поверхностный	9 баллов
Анализ выполнен только для одной модели	6 баллов
Анализ ошибок не выполнен	0 баллов

Этап 4. Инференс и демо (10 баллов)

Функция predict_letter работает корректно. Тесты на 5+ рукописных буквах. Подробный отчёт	10 баллов
Функция работает, но есть проблемы. Тесты частичные	7 баллов
Функция не работает или не тестировалась	4 балла
Функции отсутствуют	0 баллов

Оформление и выводы (5 баллов)

Ноутбук структурирован, код читаемый, графики подписаны, выводы чёткие	5 баллов
Ноутбук структурирован, но есть недочёты (короткие выводы, не все графики подписаны)	3 балла
Ноутбук не структурирован, выводы отсутствуют	0 баллов

5. Примечания

Установка библиотек

Перед началом работы установите библиотеки:

```
pip install torch torchvision tensorflow matplotlib
numpy scikit-learn pillow
```

Воспроизводимость результатов

Для воспроизводимости зафиксируйте random seed:

PyTorch:

`torch.manual_seed(42)`

TensorFlow:

```
tf.random.set_seed(42)
np.random.seed(42)
```

Особенности работы с метками

PyTorch: метки в EMNIST Letters находятся в диапазоне 1–26. При передаче в CrossEntropyLoss вычтите 1, чтобы получить диапазон 0–25.

TensorFlow: после конвертации из PyTorch метки уже находятся в диапазоне 0–25.

Сохранение моделей

PyTorch:
`torch.save(model.state_dict(), 'model_name.pth')`

TensorFlow: `model.save('model_name.h5')`

Сохранённые модели можно приложить к сдаче по желанию.

Задание, оцениваемое ментором

Введение в глубокое обучение. Итоговый проект (PJ)

Домашнее задание проверит ментор и поставит оценку. Если возникнут сложности, обращайтесь в канал модуля. Желаем успехов! ;)

Сдать домашнее задание

Сдать дз

‹ Предыдущий урок Следующий урок ›

Skillfactory — онлайн-школа IT-профессий. Онлайн-курсы по Data Science, аналитике, программированию и менеджменту © 2026

Дизайн MAX Разработка TUNE-IT